

PRODUCT
STAR.RU

SQLAlchemy



НИКОЛАЙ КОВАЛЕНКО

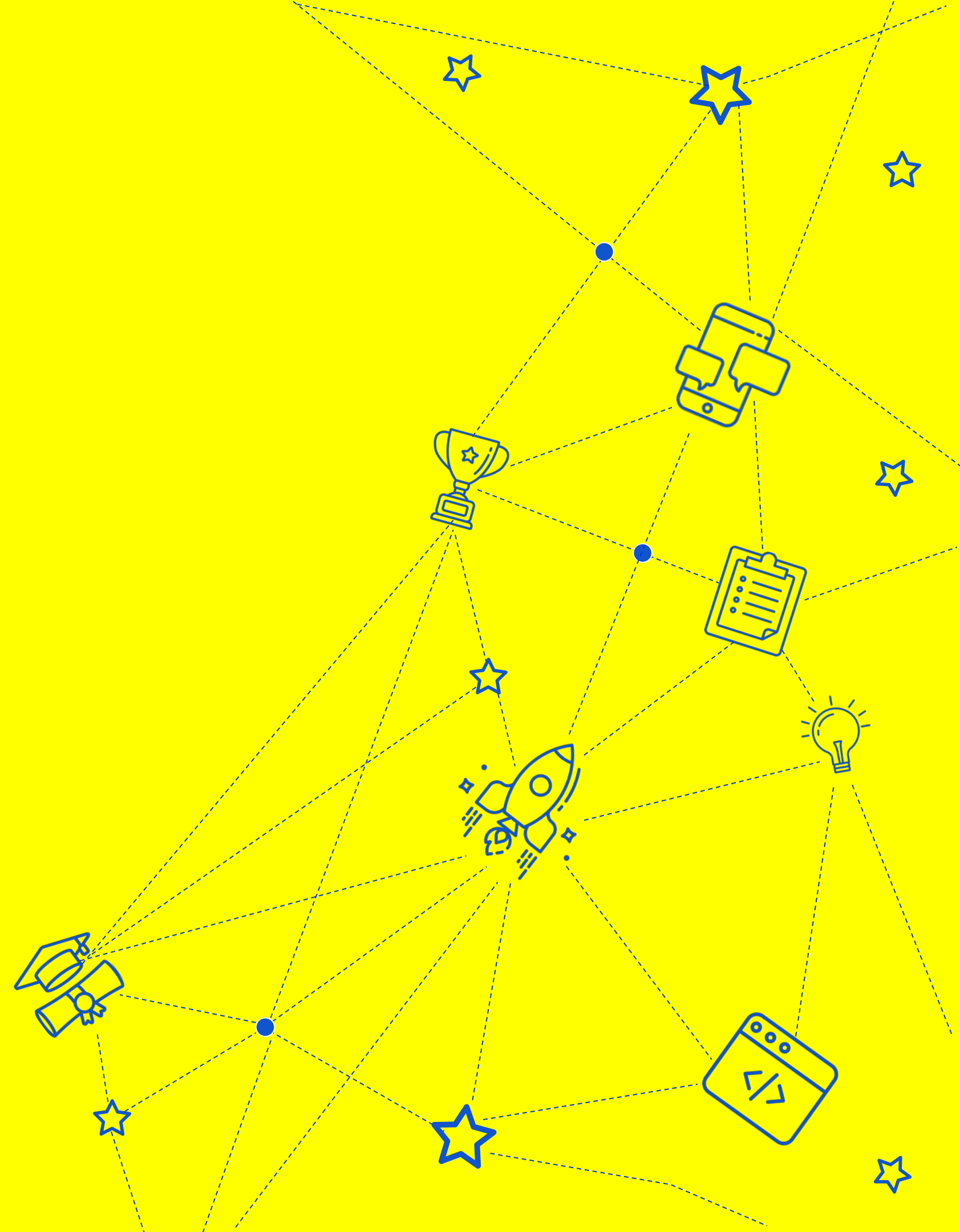
Python backend разработчик

- ZUBR Exchange – централизованная криптовалютная биржа
- Improvado – система анализа данных для маркетологов
- ВсегдаРядом – мобильное приложение для отслеживания местоположения близких
- Patient Portal – медицинский терминал самообслуживания

Содержание урока

- ★ Знакомство с ORM
- ★ Как сделать описание таблицы
- ★ Операции над таблицами
- ★ Делаем более сложные запросы

Знакомство с ORM



Object-Relational Mapping –

способ доступа к реляционной базе данных
с помощью объектно-ориентированного
языка (не SQL)

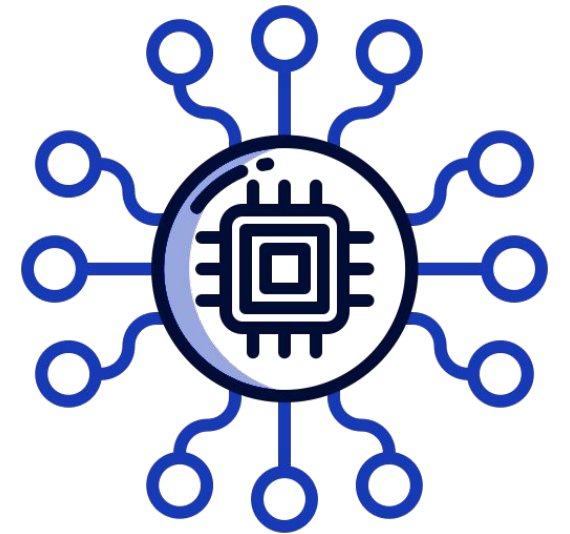


Компиляция

```
session.query(Movie).filter(Movie.release_date > date(2015, 1, 1)).limit(2)
```



```
SELECT movies.id, movies.title, movies.release_date  
FROM movies  
WHERE movies.release_date > "2015-01-01"  
LIMIT 2
```

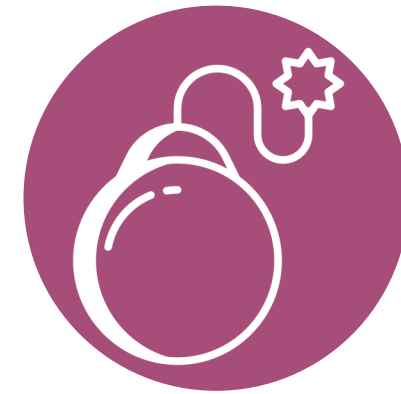


Преимущества и недостатки ORM



Что дает использование ORM по сравнению с «сырым» SQL?

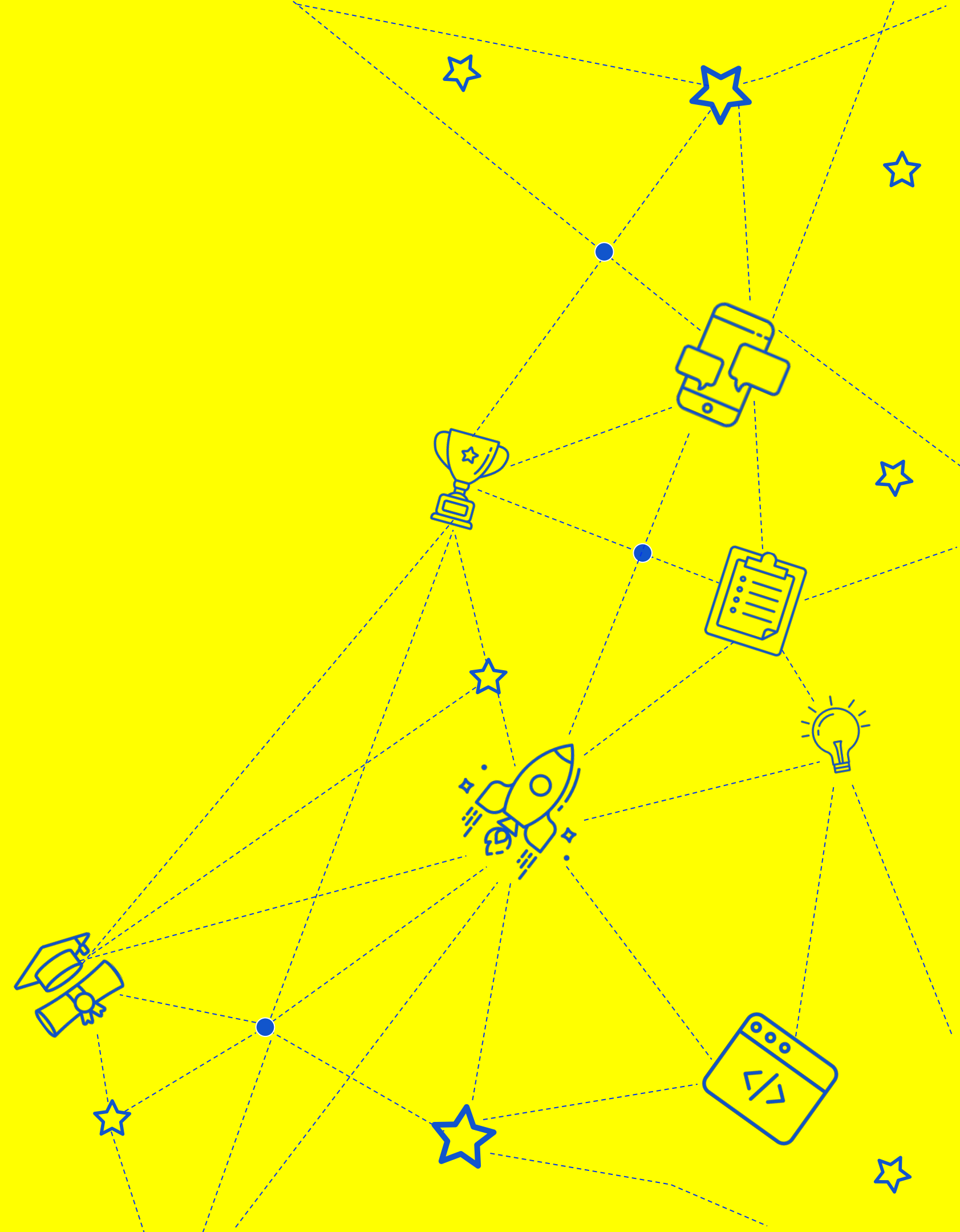
- ★ Пишем меньше кода
- ★ Единообразие кода независимо от базы данных
- ★ Кэширование запросов – уменьшение нагрузки на БД



Проблемы ORM

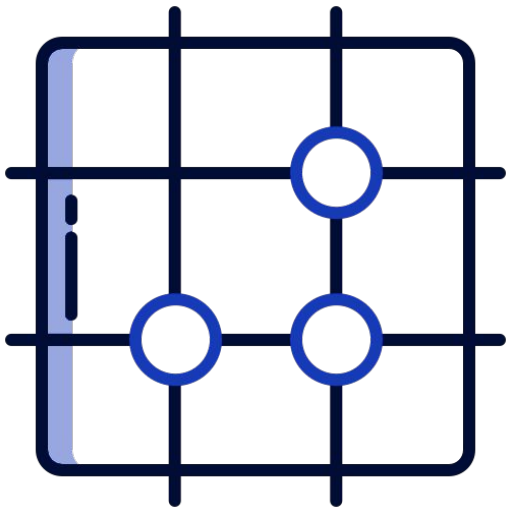
- ★ Производительность

Как сделать описание таблицы



Типы полей модели

Тип данных	Тип данных Python	Описание
Integer	int	целые числа
String	str	строка
Float	float	тип для чисел с плавающей точкой
Decimal	decimal.Decimal	тип данных Decimal
Boolean	bool	булевый тип
Date	datetime.date	дата
DateTime	datetime.datetime	дата и время
Text	str	строки произвольной длины



SQLAlchemy =<1.4

```
from sqlalchemy import Column
from sqlalchemy import ForeignKey
from sqlalchemy import Integer
from sqlalchemy import String
from sqlalchemy.orm import declarative_base
```

```
Base = declarative_base()
```

```
class User(Base):
    __tablename__ = "user_account"
    id = Column(Integer, primary_key=True)
    name = Column(String(30))
    fullname = Column(String, nullable=True)

    def __repr__(self):
        return f"User(fullname={self.fullname!r})"
```



SQLAlchemy >2.0

```
from sqlalchemy.orm import DeclarativeBase
from sqlalchemy.orm import Mapped
from sqlalchemy.orm import mapped_column

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "user_account"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(30))
    fullname: Mapped[Optional[str]]

    def __repr__(self) -> str:
        return f"User(fullname={self.fullname!r})"
```



Объявление связи между таблицами

```
class User(Base):
    ...
    addresses = relationship(
        "Address", back_populates="user", cascade="all, delete-orphan"
    )
    def __repr__(self) -> str:
        return f"User(fullname={self.fullname!r})"

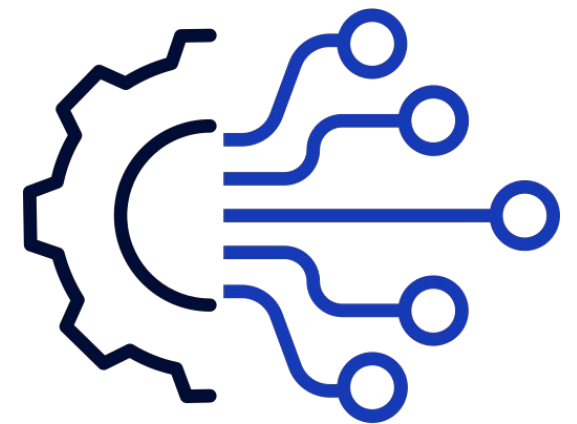
class Address(Base):
    __tablename__ = "address"
    id = Column(Integer, primary_key=True)
    email_address = Column(String)
    user_id = Column(Integer, ForeignKey("user_account.id"))
    user = relationship("User", back_populates="addresses")
    def __repr__(self) -> str:
        return f"Address(email_address={self.email_address!r})"
```



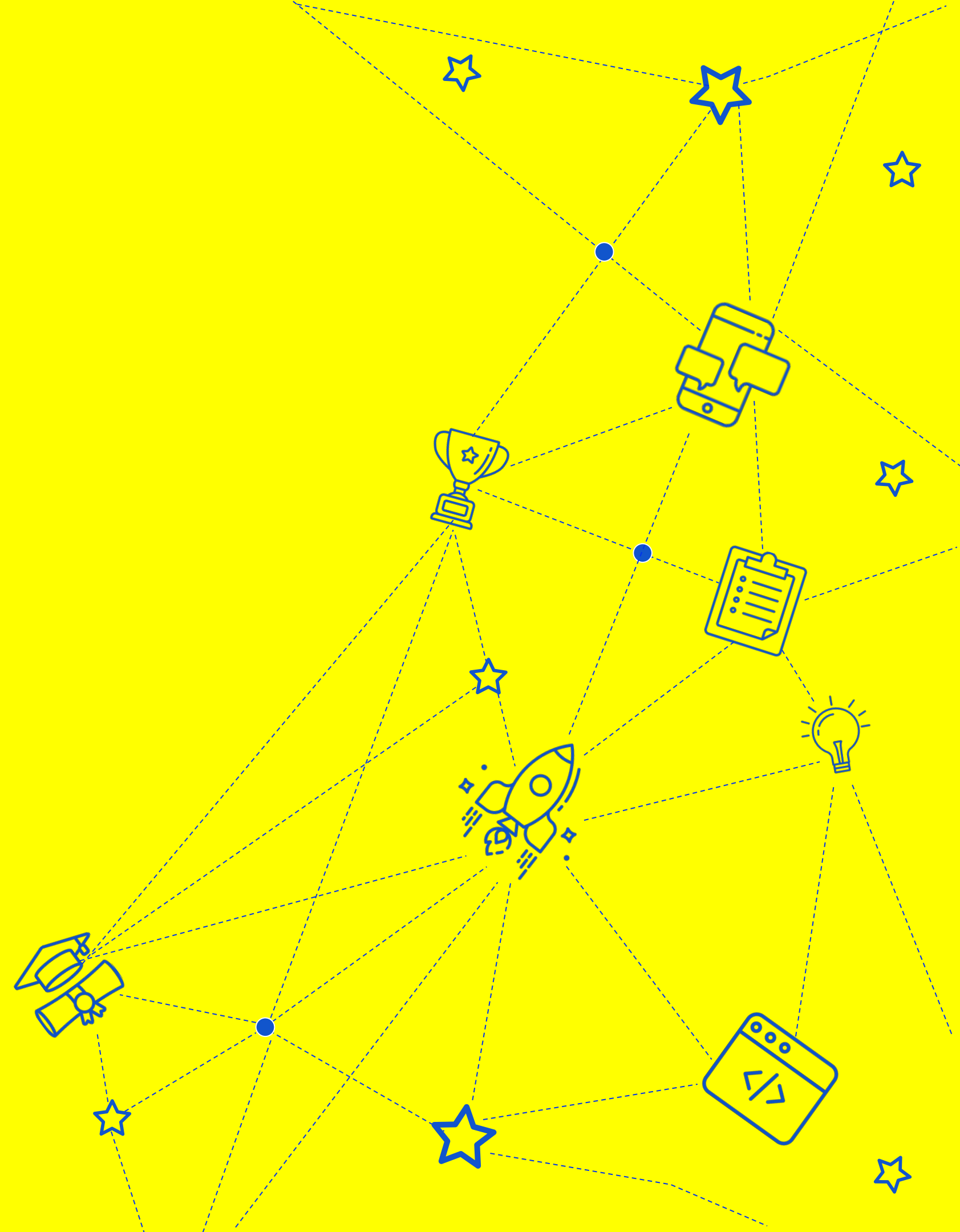
Генерация схемы БД

```
from sqlalchemy import create_engine
engine = create_engine("sqlite://", echo=True)
...
Base.metadata.create_all(engine)
```

```
CREATE TABLE user_account (
    id INTEGER NOT NULL,
    name VARCHAR(30) NOT NULL,
    fullname VARCHAR,
    PRIMARY KEY (id)
)
CREATE TABLE address (
    id INTEGER NOT NULL,
    email address VARCHAR NOT NULL,
    user_id INTEGER NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (user_id) REFERENCES user_account (id)
)
```



Операции над таблицами



Добавляем SQLAlchemy в проект

\$ pip install -U Flask-SQLAlchemy

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

# create the extension and the app
db = SQLAlchemy()
app = Flask(__name__)
# configure the SQLite database
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///project.db"
# initialize the app with the extension
db.init_app(app)

# create the tables
with app.app_context():
    db.create_all()

db.session.execute(...)
```



CRUD-операции



C

create



R

read



U

update



D

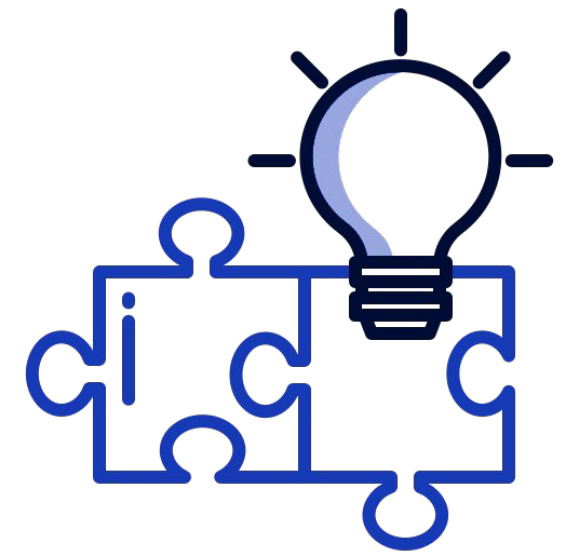
delete

Create

```
from yourapplication.models import User
```

```
user = User(  
    name="sandy",  
    fullname="Sandy Cheeks",  
)  
db.session.add(user)  
db.session.commit()
```

```
INSERT INTO user_account (name, fullname) VALUES (?, ?) RETURNING  
id  
[...] ('sandy', 'Sandy Cheeks')
```

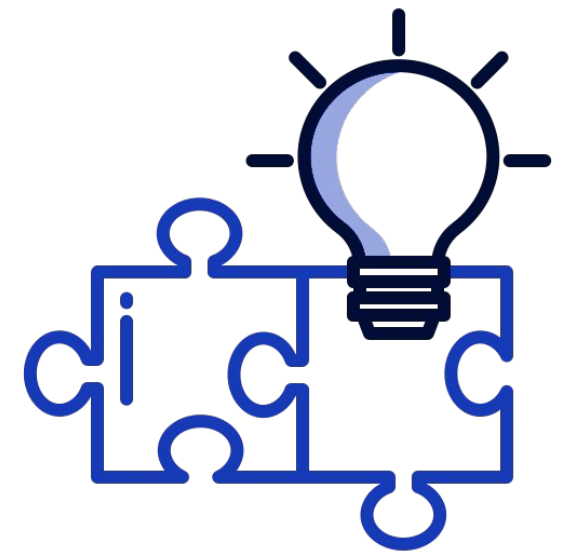


Create

...

```
user = User(
    name="sandy",
    fullname="Sandy Cheeks",
    addresses=[
        Address(email_address="sandy@sqlalchemy.org"),
    ],
)
db.session.add(user)
db.session.commit()
```

```
INSERT INTO user_account (name, fullname) VALUES (?, ?) RETURNING id
[...]( 'sandy', 'Sandy Cheeks' )
INSERT INTO address (email_address, user_id) VALUES (?, ?) RETURNING id
[...]( 'sandy@sqlalchemy.org', 1 )
```

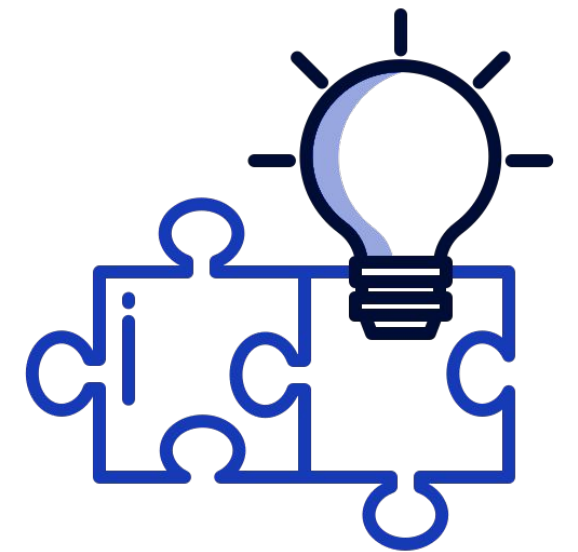


Create API

```
from yourapplication.models import User
...

@app.route("/users/create", methods=["POST"])
def user_create():
    user = User(
        name=request.form["name"],
        fullname=request.form["fullname"],
    )
    db.session.add(user)
    db.session.commit()

    return redirect(url_for("user_detail", id=user.id))
```



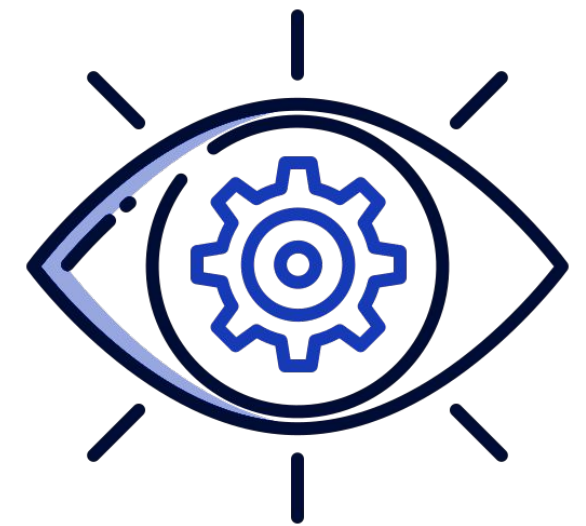
Read

```
from yourapplication.models import User
```

```
# найти всех пользователей
```

```
result = db.session.execute(  
    db.select(User)  
)
```

```
SELECT user_account.id, user_account.name,  
user_account.fullname  
FROM user_account
```



Read

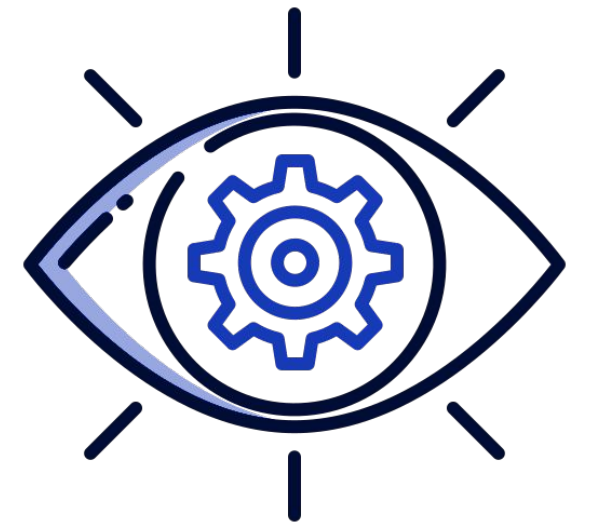
```
from yourapplication.models import User
```

```
# найти пользователей с именами spongebob и sandy
```

```
query = db.select(User).filter(  
    User.name.in_(["spongebob", "sandy"])  
)
```

```
result = db.session.execute(query)
```

```
SELECT user_account.id, user_account.name, user_account.fullname  
FROM user_account  
WHERE user_account.name IN (?, ?)  
[...] ('spongebob', 'sandy')
```

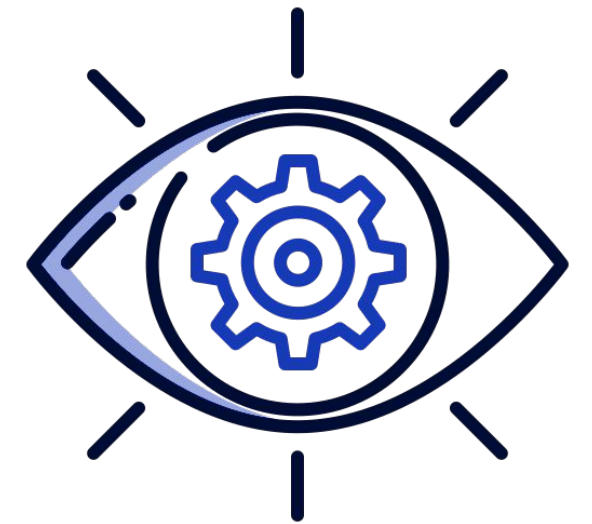


Read API

```
from yourapplication.models import User
...

@app.route("/users")
def user_list():
    query = db.select(User).order_by(User.username)
    users = db.session.execute(query).scalars()
    return render_template("user/list.html", users=users)

@app.route("/user/<int:id>")
def user_detail(id):
    user = db.get_or_404(User, id)
    return render_template("user/detail.html", user=user)
```



Update

```
from yourapplication.database import db_session

...
user.name = "patrick"
user.addresses.append(Address(email_address="ptrck@web.org"))
db.session.commit()

...
UPDATE user_account SET name=? WHERE user_account.id = ?
[...] ('patrick', 1)
INSERT INTO address (email_address, user_id) VALUES (?, ?)
[...] ('patrick@web.org', 1)
COMMIT
```



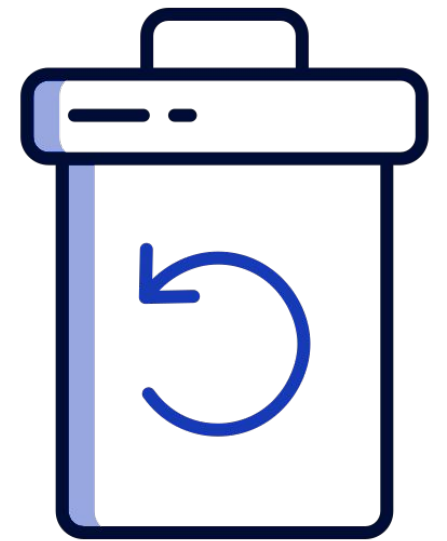
Delete

```
...
```

```
db.session.delete(address)
```

```
db.session.commit()
```

```
DELETE FROM address WHERE address.id = ?
```

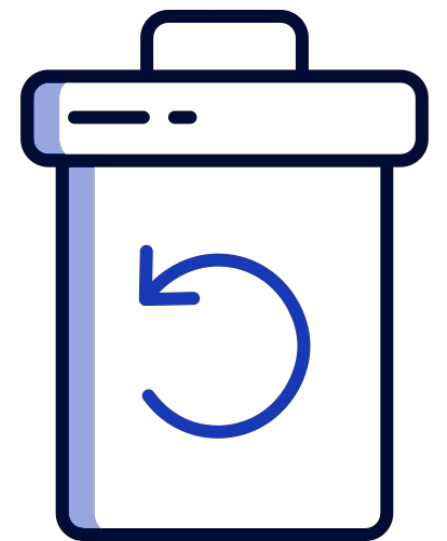


Delete API

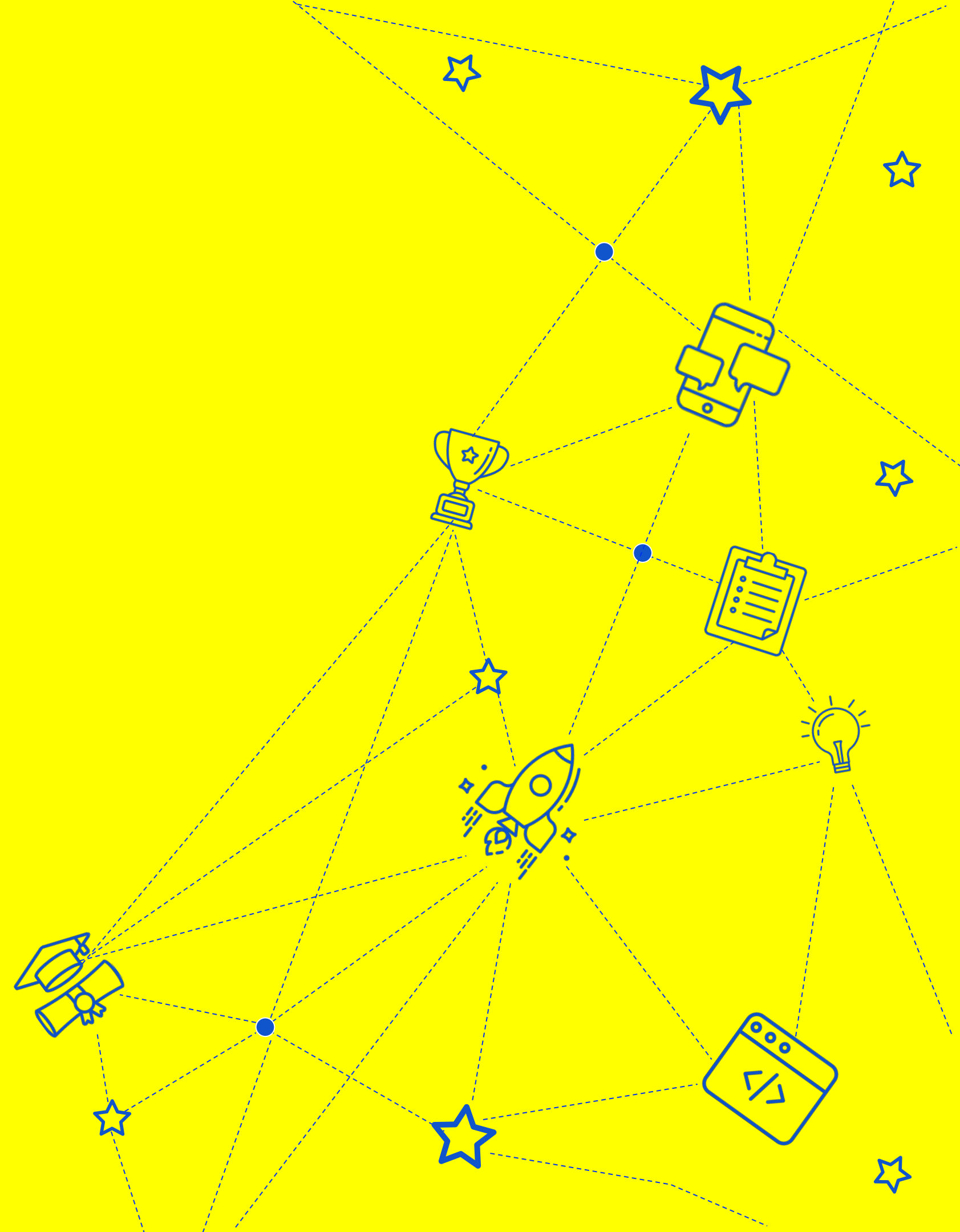
```
from yourapplication.models import User
...

@app.route("/user/<int:id>/delete", methods=["POST"])
def user_delete(id):
    user = db.get_or_404(User, id)
    db.session.delete(user)
    db.session.commit()

    return redirect(url_for("user_list"))
```



**Делаем
более сложные
запросы**

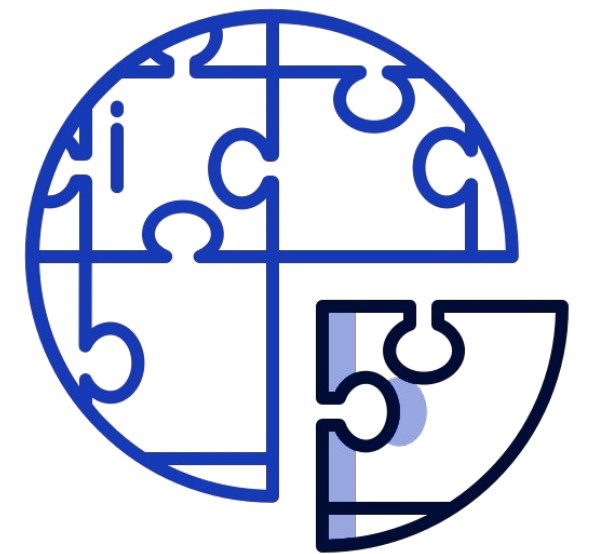


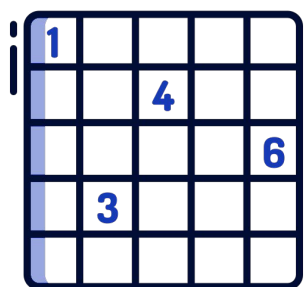
SQL JOIN B SQLAlchemy

```
from yourapplication.database import db_session
from yourapplication.models import Address, User
```

```
query = (
    db.select(Address)
    .join(Address.user)
    .where(User.name == "sandy")
)
```

```
SELECT address.id, address.email_address, address.user_id
FROM address JOIN user_account ON user_account.id =
address.user_id
WHERE user_account.name = ?
[...] ('sandy')
```

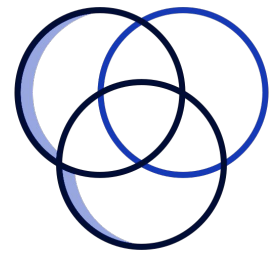




Объединяем больше таблиц

```
db.select (User) .join (User.orders) .join (Order.items)
```

```
SELECT user_account.id, user_account.name, user_account.fullname  
FROM user_account  
JOIN user_order ON user_account.id = user_order.user_id  
JOIN order_items AS order_items_1 ON user_order.id =  
order_items_1.order_id  
JOIN item ON item.id = order_items_1.item_id
```

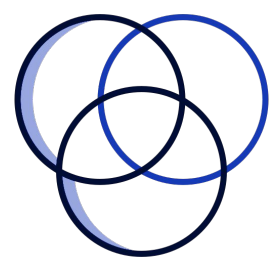


Агрегация и сортировка

```
from sqlalchemy import func, desc
```

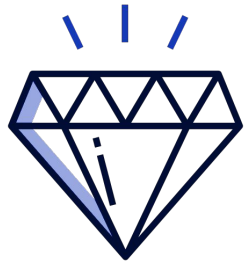
```
query = (  
    db.select(Address.user_id, func.count(Address.id).label("addresses_count"))  
    .group_by("user_id")  
    .order_by("user_id", desc("addresses_count"))  
)
```

```
SELECT address.user_id, count(address.id) AS addresses_count  
FROM address GROUP BY address.user_id ORDER BY address.user_id, addresses_count  
DESC
```



Функции агрегации

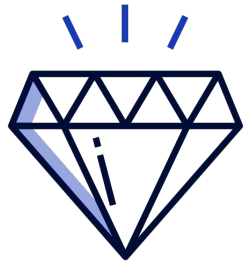
Функция 	Описание 
count()	Возвращает количество элементов выборки
avg()	Возвращает среднее арифметическое выборки
sum()	Возвращает сумму элементов выборки
min() / max()	Возвращает минимальный или максимальный элемент выборки



Полезные методы Query

```
q: flask_sqlalchemy.query.Query = db.session.query(User)
```

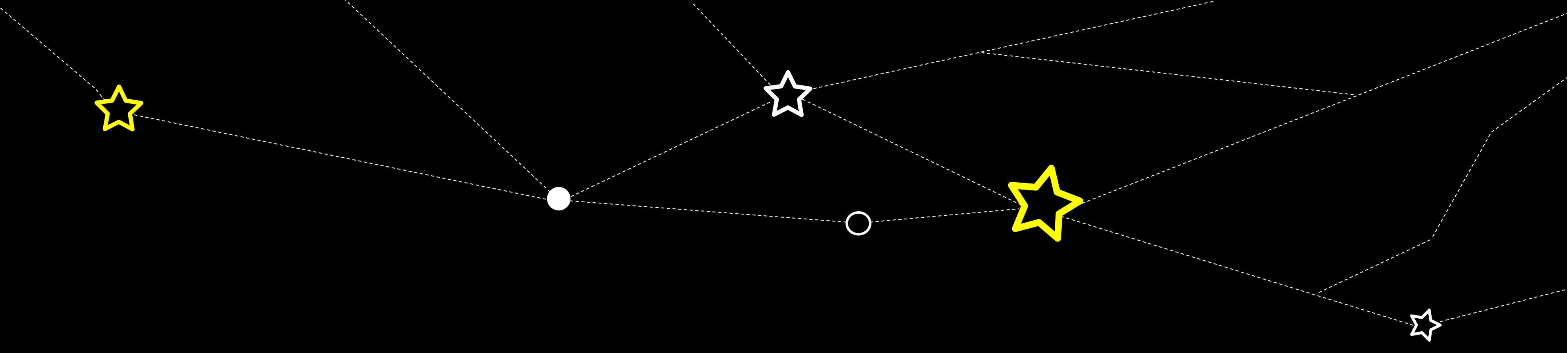
Метод 	Описание 
all()	Возвращает результат запроса (представленный flask_sqlalchemy.BaseQuery) в виде списка
count()	Возвращает количество записей в запросе
first()	Возвращает первый результат запроса или None, если в нем нет строк
get_or_404(pk)	Возвращает объект, который соответствует данному первичному ключу или ошибку 404, если объект не найден



Полезные методы Query

```
q: flask_sqlalchemy.query.Query = db.session.query(User)
```

Метод 	Описание 
limit(limit)	Добавляет к запросу SQL-оператор LIMIT
offset(offset)	Добавляет к запросу SQL-оператор OFFSET
get()	Возвращает объект, который соответствует данному первичному ключу или None, если объект не найден.
one()	Возвращает ровно один результат из выборки, в противном случае генерирует исключение



СПАСИБО ЗА ВНИМАНИЕ

Николай Коваленко



[nikolay_kovalenko_91](https://t.me/nikolay_kovalenko_91)



getmentor.dev/mentor/nikolay-kovalenko-1023



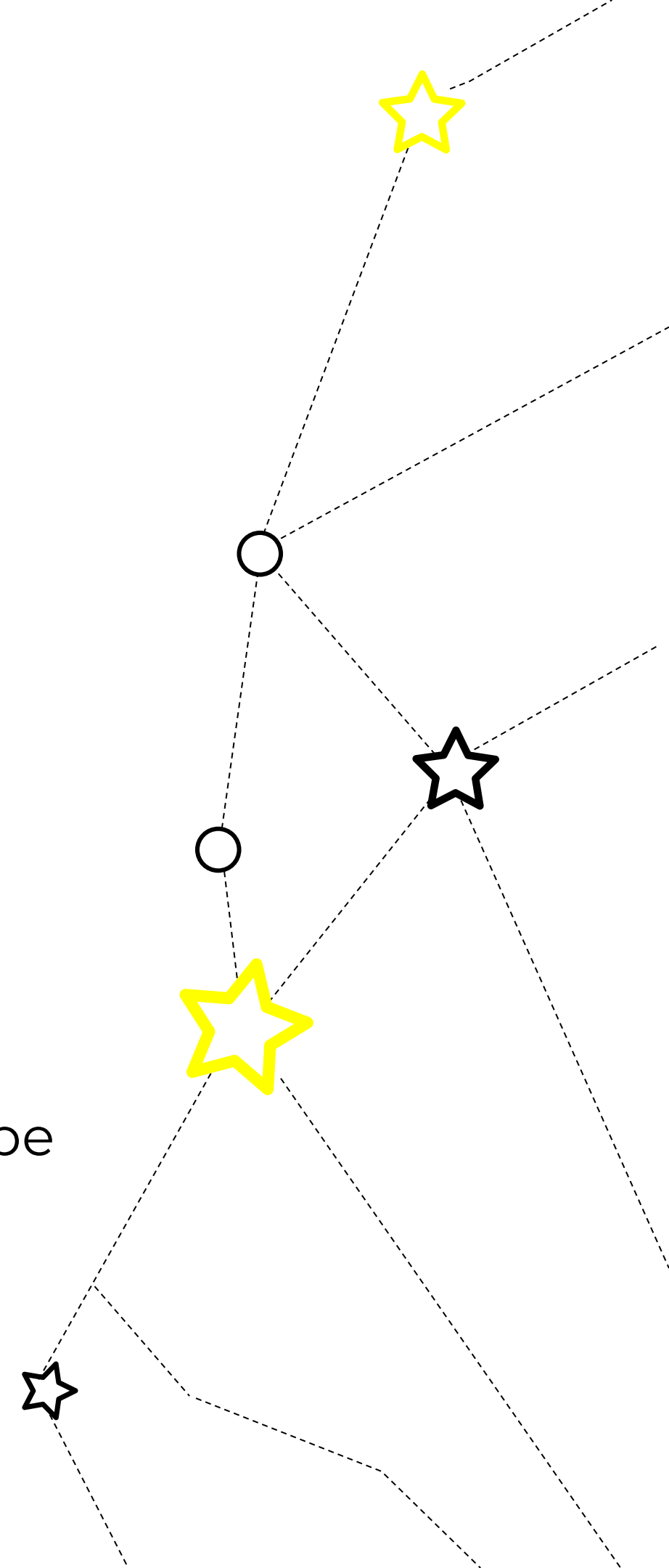
solvery.io/ru/mentor/nikolay_kovalenko

Домашнее задание



Проект Books

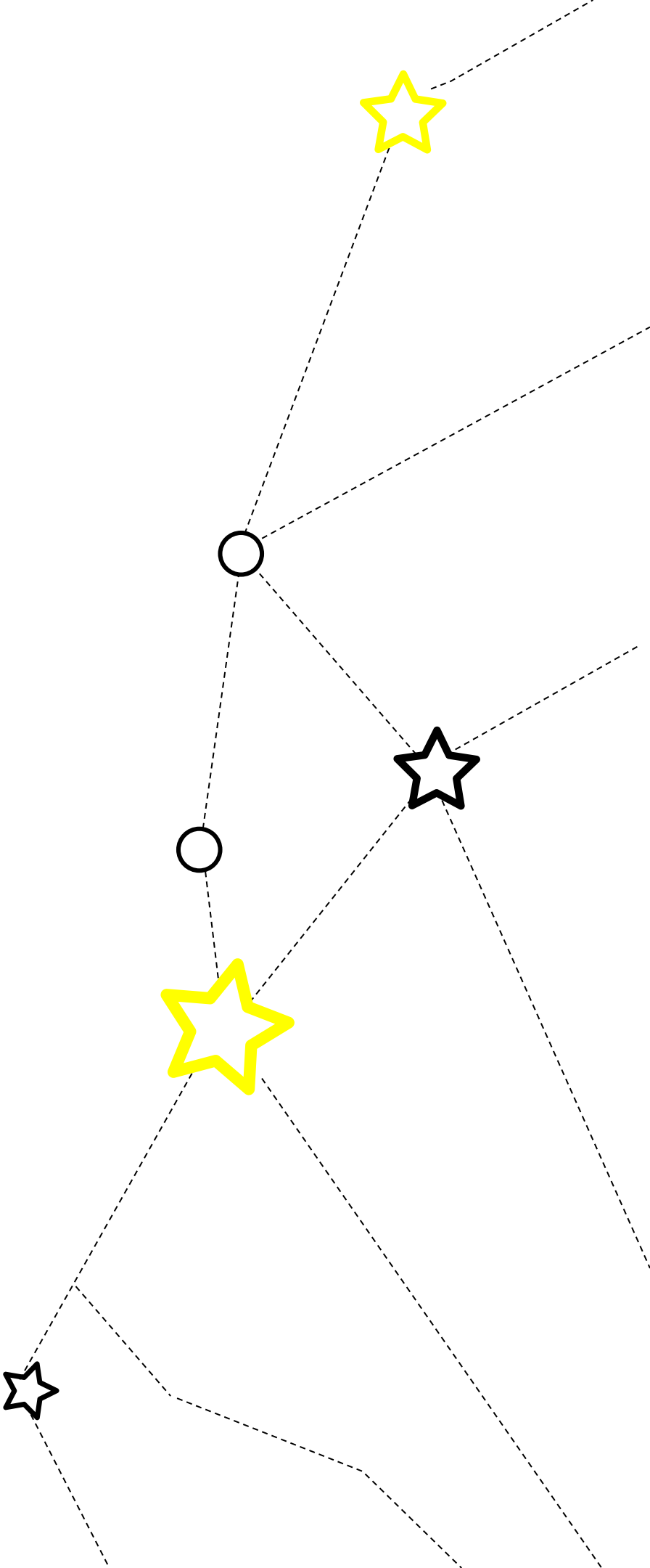
- 1 Подключить базу данных к новому проекту
- 2 Создать модели Book и Genre
- 3 Вывести 15 последних записей из Book на главную страницу приложения
- 4 Сделать возможность перехода с главной на страницу жанра книги (Genre), где можно увидеть все записи из Book выбранном в жанре



Домашнее задание

Поля модели Book

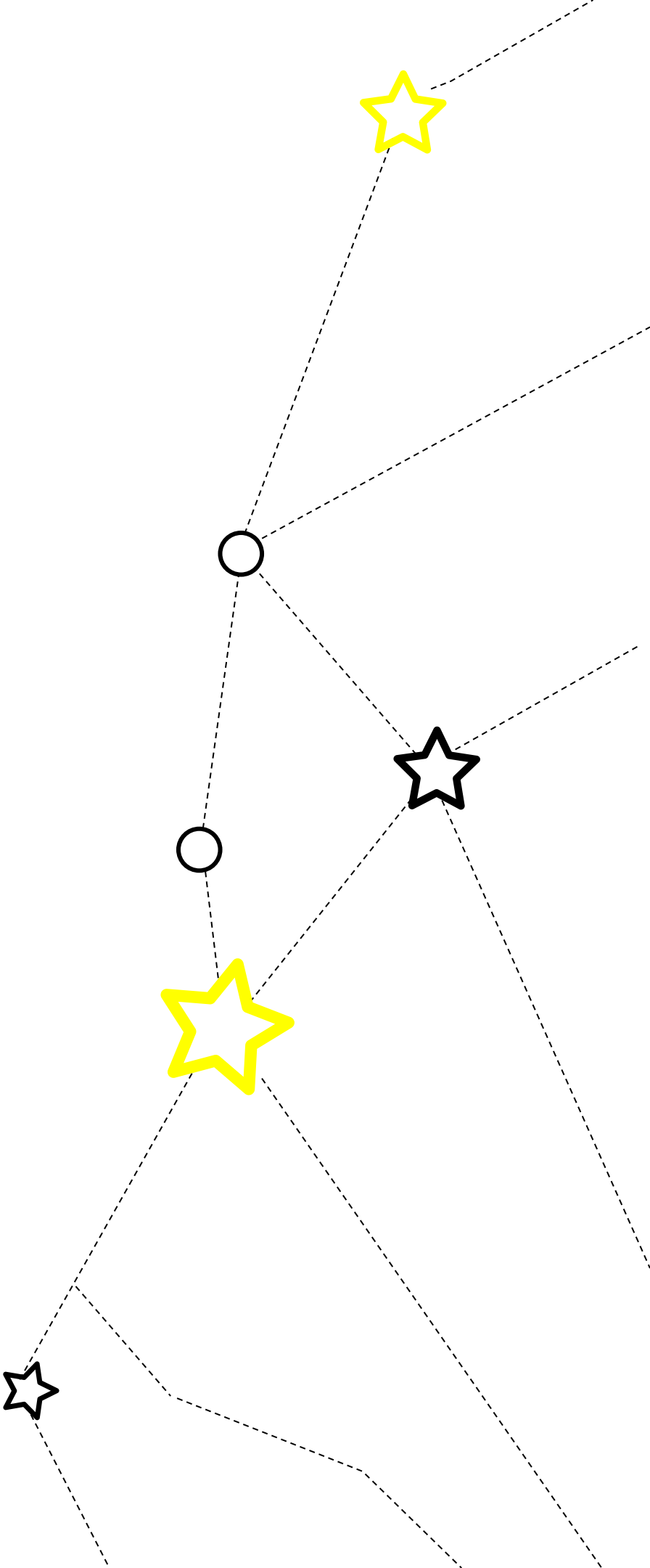
Название поля	Имя поля	Тип
Дата публикации	publication_date	Дата (текущая дата добавляется автоматически при записи в БД)
Название книги	title	Строка
Автор	author	Строка
Аннотация	annotation	Текст. Поле опционально
	genre_id	Foreign Key для связи с моделью Genre (один-ко-многим)



Домашнее задание

Поля модели Genre

Название поля	Имя поля	Тип
Название	name	Строка
Описание	description	Текст. Поле опционально



Дополнительное задание*



Проект Books

- 1 Добавить к модели Book поле с именем "is_read", тип - boolean
- 2 В отображении объектов модели Book на главной странице добавить html-чекбокс, отображающий свойство "is_read" (прочитана книга или нет), и кнопку "Сохранить".
- 3 При нажатии на чекбокс и кнопку "Сохранить" текущий статус книги (прочитана или нет) должен сохраняться в БД вызовом к API приложения методом POST.

***необязательное**

